作业 _Week10

20244227004 蔡博凯

2024 年 11 月 6 日

第一题

同一样本空间中的集合 X 与 Z 之间的距离可通过“豪斯多夫距离”(Hausdorff distance)计算:

$$\mathrm{dist_H}(X,Z)=\max\left(\mathrm{dist_h}(X,Z),\mathrm{dist_h}(Z,X)\right),$$

其中

$$\mathrm{dist_h}(X,Z)=\max_{\boldsymbol{x}\in X}\min_{\boldsymbol{z}\in Z}\|\boldsymbol{x}-\boldsymbol{z}\|_2.$$

试证明: 豪斯多夫距离满足距离度量的四条基本性质.

表 9.1 西瓜数据集 4.0

编号	密度	含糖率	编号	密度	含糖率	编号	密度	含糖率
1	0.697	0.460	11	0.245	0.057	21	0.748	0.232
2	0.774	0.376	12	0.343	0.099	22	0.714	0.346
3	0.634	0.264	13	0.639	0.161	23	0.483	0.312
4	0.608	0.318	14	0.657	0.198	24	0.478	0.437
5	0.556	0.215	15	0.360	0.370	25	0.525	0.369
6	0.403	0.237	16	0.593	0.042	26	0.751	0.489
7	0.481	0.149	17	0.719	0.103	27	0.532	0.472
8	0.437	0.211	18	0.359	0.188	28	0.473	0.376
9	0.666	0.091	19	0.339	0.241	29	0.725	0.445
10	0.243	0.267	20	0.282	0.257	30	0.446	0.459

解答

豪斯多夫距离定义

$$\mathrm{dist}_H(X,Z)=\max\left(\max_{x\in X}\min_{z\in Z}\|x-z\|_2,\max_{z\in Z}\min_{x\in X}\|z-x\|_2\right).$$

1. 非负性

对于任意集合 X 和 Z，有 $\mathrm{dist}_H(X,Z)\geq 0$

其中每一个 $\|z-x\|_2,\|x-z\|_2\geq 0$，因此 $\mathrm{dist}_H(X,Z)\geq 0$。

2. 同一性

$\mathrm{dist}_H(X,Z)=0$ 当且仅当 X=Z

必要性：若 $\mathrm{dist}_H(X,Z)=0$，则 $X=Z$

假设 $\mathrm{dist}_H(X,Z)=0$，则

$$\mathrm{dist}_h(X,Z)=0 \quad 且 \quad \mathrm{dist}_h(Z,X)=0.$$

这意味着：

(1). 对于每个 $x \in X$，存在 $z \in Z$ 使得 $\|x - z\|_2 = 0$，即 $x = z$；

(2). 对于每个 $z \in Z$，存在 $x \in X$ 使得 $\|z - x\|_2 = 0$，即 $z = x$。

因此，$X \subseteq Z$ 且 $Z \subseteq X$，从而 $X = Z$。

充分性：若 $X = Z$，则 $\mathrm{dist}_H(X, Z) = 0$

若 $X = Z$，则对于任意 $x \in X$（即 $x \in Z$），有

$$\min_{z \in Z} \|x - z\|_2 = 0.$$

因此，$\mathrm{dist}_h(X, Z) = 0$。同理，$\mathrm{dist}_h(Z, X) = 0$。从而

$$\mathrm{dist}_H(X, Z) = \max\left(\mathrm{dist}_h(X, Z), \mathrm{dist}_h(Z, X)\right) = 0.$$

综上所述，豪斯多夫距离的同一性得证，即

$$\mathrm{dist}_H(X, Z) = 0 \iff X = Z.$$

3. 对称性

定义中有

$$\mathrm{dist}_H(X, Z) = \max\left(\max_{x \in X} \min_{z \in Z} \|x - z\|_2, \max_{z \in Z} \min_{x \in X} \|z - x\|_2\right).$$

交换变量后

$$\mathrm{dist}_H(Z, X) = \max\left(\max_{z \in Z} \min_{x \in X} \|z - x\|_2, \max_{x \in X} \min_{z \in Z} \|x - z\|_2\right).$$

可以看出交换 X 和 Z 的位置不会改变值，因此 $\mathrm{dist}_H(X, Z) = \mathrm{dist}_H(Z, X)$。

4. 直递性

设 X、Y 和 Z 为样本空间中的集合。需要证明

$$\mathrm{dist}_H(X, Z) \leq \mathrm{dist}_H(X, Y) + \mathrm{dist}_H(Y, Z).$$

我们分别考虑 $\mathrm{dist}_h(X, Z)$ 和 $\mathrm{dist}_h(Z, X)$。考虑以下表达式：

$$\max_{x \in X} \min_{z \in Z} \|x - z\|_2.$$

引入一个中间集合 Y，并注意到对于任何 $x \in X$ 和 $z \in Z$，通过三角不等式有：

$$\|x - z\|_2 \leq \|x - y\|_2 + \|y - z\|_2,$$

其中 $y \in Y$ 是一个中间点。因此，对于每个 $x \in X$，有

$$\min_{z \in Z} \|x - z\|_2 \leq \min_{y \in Y} \|x - y\|_2 + \min_{z \in Z} \|y - z\|_2.$$

对所有 $x \in X$ 取最大值，得到

$$\max_{x \in X} \min_{z \in Z} \|x - z\|_2 \leq \max_{x \in X} \min_{y \in Y} \|x - y\|_2 + \max_{y \in Y} \min_{z \in Z} \|y - z\|_2. \tag{a}$$

考虑以下表达式：

$$\max_{z \in Z} \min_{x \in X} \|z - x\|_2.$$

通过类似的推理，得到

$$\max_{z \in Z} \min_{x \in X} \|z - x\|_2 \leq \max_{z \in Z} \min_{y \in Y} \|z - y\|_2 + \max_{y \in Y} \min_{x \in X} \|y - x\|_2. \tag{b}$$

综合以上两条不等式 (a)、(b) 取上述两式的最大值，得到

$$\mathrm{dist}_H(X, Z) \leq \mathrm{dist}_H(X, Y) + \mathrm{dist}_H(Y, Z).$$

至此，直递性得证。

综上，豪斯多夫距离 $\mathrm{dist}_H(X, Z)$ 满足距离度量的四条基本性质，因此它是一个有效的距离度

第二题

试编程实现 k 均值算法, 设置三组不同的 k 值、三组不同初始中心点, 在西瓜数据集 4.0 上进行实验比较, 并讨论什么样的初始中心有利于取得好结果.

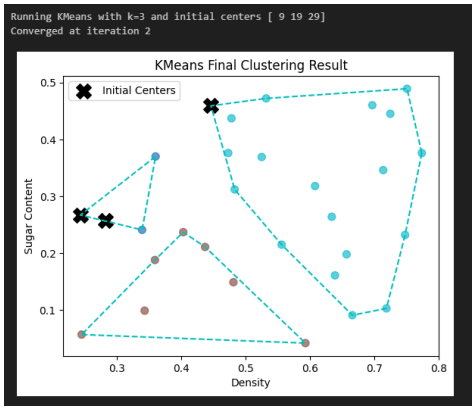
解答

1. 代码概述

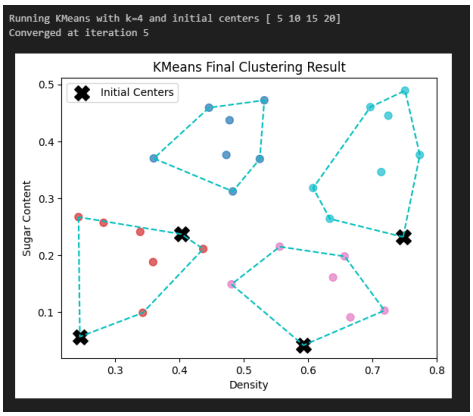
代码实现了 KMeans 聚类算法,并支持不同簇数和初始中心点的训练。KMeans 类包括聚类过程中的样本分类、簇中心更新和可视化功能。代码针对三组不同的 k 值和初始中心点集合执行训练，并绘制聚类结果。

可在附录查看完整代码，请参考 KMeans。

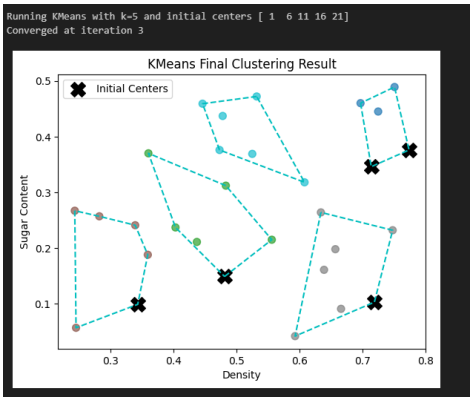
2. 运行结果



(a) k=3



(b) k=4



(c) k=5

图 1: 运行结果

可以观察到，初始点的设置与 k 的选择对训练轮数产生影响，如果没有设

置好初始中心，可能会降低模型收敛速度。因此选择合适的初始点很重要，经过查找资料和总结，可以得出理想的初始点应满足以下几个条件：

(1). 均匀分布：初始簇中心应尽量分布在数据空间的不同区域，避免过于集中在某一位置，以确保簇间有足够的区别。

(2). 代表性：选择能代表数据集结构的点，通常是位于数据分布边缘或者较为典型的点，避免选择噪声或极端异常点。

(3). 远离其他点：选择的初始点应相互距离较远，避免簇的重叠，帮助聚类算法有效地区分不同的簇。

(4). 覆盖数据密集区域：初始点应覆盖数据集中的高密度区域，避免将初始点放在数据稀疏或无关紧要的地方。

第三题

基于 DBSCAN 的概念定义, 若 $\boldsymbol{x}$ 为核心对象, 由 $\boldsymbol{x}$ 密度可达的所有样本构成的集合为 X. 试证明: X 满足连接性(9.39)与最大性(9.40).

连接性(connectivity): $\boldsymbol{x_i} \in C, \boldsymbol{x_j} \in C \Rightarrow \boldsymbol{x_i}$ 与 $\boldsymbol{x_j}$ 密度相连 (9.39)

最大性(maximality): $\boldsymbol{x_i} \in C, \boldsymbol{x_j}$ 由 $\boldsymbol{x_i}$ 密度可达 $\Rightarrow \boldsymbol{x_j} \in C$ (9.40)

解答

1. 证明连接性

对于集合 X 中的任意两点 x_i 和 x_j, 由于 x_i 与核心对象 x 是密度可达的，x_j 与核心对象 x 是也密度可达的。说明对 x_i 和 x_j，存在 x_k 使得 x_i 和 x_j 均由 x_k 密度可达，由密度相连的性质可知，x_i 和 x_j 密度相连，故得证。

2. 证明最大性

使用反证法，假设 $x_j \notin X$，说明 x 与 x_j 是密度不可达的，也即不存在样本序列 $1(x, .., x_j)$。由于 $x_i \in X$, 说明 x 与 x_i 是密度可达的, 也即存在样本序列 $2(x, .., x_i)$。又因为 x_j 由 x_i 密度可达，即存在样本序列 $3(x_i, .., x_j)$。由于序列 2 和序列 3 结合可得到序列 1，说明存在这样的序列 1，与假设矛盾，故 $x_j \in X$ 得证。

结论：由核心对象 x 密度可达的所有样本集合 X 满足连接性和最大性。

参考文献

[1] https://github.com/hanmq/MachineLearning_Zhouzhihua_ProblemSets

A KMeans

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.spatial import ConvexHull

class KMeans(object):
    def __init__(self, k):
        """初始化KMeans对象，k为簇的数量"""
        self.k = k

    def fit(self, X, initial_centroid_index=None, max_iters=10,
        seed=16, plt_process=False):
        """
        训练KMeans模型，找到最终的中心点，并返回样本的分类标签

        参数:
        X : ndarray
            数据集，形状为(m, n)，m为样本数，n为特征数
        initial_centroid_index : ndarray, optional
            初始簇中心的索引，若为None则随机初始化
        max_iters : int, optional
            最大迭代次数，默认10次
        seed : int, optional
```

```
22              随机种子，默认为16
23          plt_process : bool, optional
24              是否可视化过程，默认为False
25
26          返回:
27          centroid : ndarray
28              最终的簇中心
29          idx : ndarray
30              每个样本所属簇的索引
31          """
32          m, n = X.shape
33          # 如果没有提供初始中心点索引，随机初始化
34          if initial_centroid_index is None:
35              np.random.seed(seed)
36              initial_centroid_index = np.random.choice(m, self.k,
                    replace=False)
37          centroid = X[initial_centroid_index, :]
38          idx = np.zeros(m)
39          for i in range(max_iters):
40              # 找到每个点距离最近的簇中心
41              idx = self.find_closest_centroids(X, centroid)
42
43              # 重新计算簇中心
44              new_centroid = self.compute_centroids(X, idx)
45
46              # 如果簇中心没有变化，提前结束迭代
47              if np.allclose(centroid, new_centroid):
48                  print(f'Converged at iteration {i}')
49                  break
50
51              centroid = new_centroid
52
53          # 训练结束后绘制最终的分类结果
```

```
54          self.plot_final(X, idx, initial_centroid_index)
55
56          return centroid, idx
57
58      def find_closest_centroids(self, X, centroid):
59          """
60          为每个样本分配到最近的簇中心
61
62          参数:
63          X : ndarray
64              样本数据集
65          centroid : ndarray
66              当前簇中心
67
68          返回:
69          idx : ndarray
70              每个样本所属的簇的索引
71          """
72          # 通过广播机制，计算样本到各个中心点的距离
73          distance = np.linalg.norm(X[:, np.newaxis, :] - centroid
                , axis=2)
74          return np.argmin(distance, axis=1)
75
76      def compute_centroids(self, X, idx):
77          """
78          根据样本分配结果，计算每个簇的新的中心点
79
80          参数:
81          X : ndarray
82              样本数据集
83          idx : ndarray
84              每个样本所属簇的索引
85
```

```
86          返回:
87          centroids : ndarray
88              计算得到的新的簇中心
89          """
90          centroids = np.zeros((self.k, X.shape[1]))
91          for i in range(self.k):
92              # 计算每个簇的中心点
93              centroids[i] = np.mean(X[idx == i], axis=0)
94          return centroids
95
96      def plot_final(self, X, idx, initial_idx):
97          """
98          绘制KMeans训练结束后的最终分类结果
99
100         参数:
101         X : ndarray
102             数据集
103         idx : ndarray
104             每个样本所属簇的索引
105         initial_idx : ndarray
106             初始簇中心的索引
107         """
108         plt.figure()
109         plt.title("KMeans Final Clustering Result")
110         plt.xlabel('Density')
111         plt.ylabel('Sugar Content')
112         # 根据簇的标签绘制样本点
113         plt.scatter(X[:, 0], X[:, 1], c=idx, cmap='tab10', s=50,
                alpha=0.7)
114         plt.scatter(X[initial_idx, 0], X[initial_idx, 1], label=
              'Initial Centers', c='black', s=200, marker='X')
115
116         # 绘制每个簇的凸包
```

```
117         for i in range(self.k):
118             X_i = X[idx == i]
119             if len(X_i) > 2:  # 至少3个点才有凸包
120                 hull = ConvexHull(X_i).vertices
121                 hull = np.append(hull, hull[0])  # 闭合凸包
122                 plt.plot(X_i[hull, 0], X_i[hull, 1], 'c--')
123 
124         plt.legend()
125         plt.show()
126 
127 if __name__ == '__main__':
128     # 加载数据集
129     data = np.loadtxt('./watermelon4_0_Ch.txt', delimiter=',')
130     # 设定不同的k值和初始中心点索引
131     k_values = [3, 4, 5]  # 三组不同的簇数
132     initial_centroids_sets = {
133         3: np.array([9, 19, 29]),  # 初始中心点集1
134         4: np.array([5, 10, 15, 20]),  # 初始中心点集2
135         5: np.array([1, 6, 11, 16, 21])  # 初始中心点集3
136     }
137     # 遍历不同的k值和相应的初始中心点
138     for k in k_values:
139         initial_centroids = initial_centroids_sets[k]
140         # 确保初始中心点的数量与k一致
141         if len(initial_centroids) != k:
142             print(f"Error: The number of initial centroids for k
                =={k} must be equal to {k}.")
143             continue
144         print(f"Running KMeans with k={k} and initial centers {
            initial_centroids}")
145         # 使用KMeans进行训练
146         KMeans(k).fit(data, initial_centroid_index=
            initial_centroids, plt_process=False, seed=24)
```